

SQL: SELECT, JOIN, GROUP BY, Subqueries

I'll use a generic **payment-system example** so you can later map it to your BPS tables yourself.

1. SELECT

SELECT is used to retrieve data.

Suppose we have:

customer

customer_id	name	city
C101	Rahul	Kolkata
C102	Amit	Delhi
C103	Neha	Mumbai

payment

payment_id	customer_id	amount	status
P1	C101	1000	SUCCESS
P2	C101	2000	FAILED
P3	C102	5000	SUCCESS

Basic query:

```
SELECT *  
FROM payment;
```

Better:

```
SELECT payment_id, customer_id, amount, status  
FROM payment;
```

Why avoid **SELECT *** ?

Because you may:

- retrieve unnecessary columns
- increase network/data transfer
- make code more dependent on schema changes

- reduce readability

For production queries, explicitly selecting required columns is generally preferable.

2. WHERE

Use **WHERE** to filter rows.

```
SELECT payment_id, amount
FROM payment
WHERE status = 'SUCCESS';
```

Result:

payment_id	amount
P1	1000
P3	5000

Multiple conditions:

```
SELECT payment_id, amount
FROM payment
WHERE status = 'SUCCESS'
AND amount > 1000;
```

3. ORDER BY

Sort the result.

```
SELECT payment_id, amount
FROM payment
ORDER BY amount DESC;
```

```
P3 → 5000
P2 → 2000
P1 → 1000
```

4. DISTINCT

Remove duplicate result values.

```
SELECT DISTINCT status
FROM payment;
```

Result:

SUCCESS
FAILED

5. LIMIT

Useful when you only need a small number of rows.

```
SELECT *  
FROM payment  
ORDER BY amount DESC  
LIMIT 5;
```

Exact syntax depends on the database.

6. JOIN

Suppose:

```
customer  
|  
| customer_id  
↓  
payment
```

We want:

Customer name + Payment amount

That's where **JOIN** comes in.

7. INNER JOIN

INNER JOIN returns rows where the join condition matches in **both tables**.

```
SELECT  
    c.customer_id,  
    c.name,  
    p.payment_id,  
    p.amount  
FROM customer c  
INNER JOIN payment p  
    ON c.customer_id = p.customer_id;
```

Result:

customer_id	name	payment_id	amount
C101	Rahul	P1	1000

C101	Rahul	P2	2000
C102	Amit	P3	5000

Notice:

C103 → Neha

has no payment, so it doesn't appear.

Mental model

Customer Payment

```

A  _____  A
B  _____  B
C  _____  ← no match

```

INNER JOIN
→ only matched rows

8. LEFT JOIN

LEFT JOIN returns **all rows from the left table**, plus matching rows from the right table.

```

SELECT
  c.customer_id,
  c.name,
  p.payment_id,
  p.amount
FROM customer c
LEFT JOIN payment p
  ON c.customer_id = p.customer_id;

```

Result:

customer_id	name	payment_id	amount
C101	Rahul	P1	1000
C101	Rahul	P2	2000
C102	Amit	P3	5000
C103	Neha	NULL	NULL

C103 appears because **customer** is the **left table**.

No matching payment → right-side columns become **NULL**.

9. RIGHT JOIN

RIGHT JOIN is the opposite.

It returns all rows from the **right table**, plus matching rows from the left.

```
SELECT
    c.name,
    p.payment_id,
    p.amount
FROM customer c
RIGHT JOIN payment p
    ON c.customer_id = p.customer_id;
```

Conceptually:

```
All payments
+
matching customers
```

In practice, many developers prefer rewriting a `RIGHT JOIN` as a `LEFT JOIN` by swapping table order because it is often easier to read.

For example:

```
-- RIGHT JOIN
customer RIGHT JOIN payment

-- Equivalent LEFT JOIN
payment LEFT JOIN customer
```

10. INNER vs LEFT vs RIGHT

Memorize this:

```
INNER JOIN
→ only matches

LEFT JOIN
→ everything from LEFT + matches from RIGHT

RIGHT JOIN
→ everything from RIGHT + matches from LEFT
```

Visual

```
INNER
    A ∩ B

LEFT
    A + matching B

RIGHT
    B + matching A
```

11. Why LEFT JOIN Is Very Important

Suppose interviewer asks:

"Find customers who haven't made any payment."

You need `LEFT JOIN`.

```
SELECT
  c.customer_id,
  c.name
FROM customer c
LEFT JOIN payment p
  ON c.customer_id = p.customer_id
WHERE p.payment_id IS NULL;
```

Result:

C103 | Neha

This is a very common interview query.

12. JOIN and One-to-Many Relationships

Suppose:

```
Customer C101
  ↓
Payments P1, P2
```

Then:

```
SELECT c.name, p.payment_id
FROM customer c
JOIN payment p
  ON c.customer_id = p.customer_id;
```

returns:

Rahul | P1
Rahul | P2

So one customer appears multiple times.

That's not a duplicate caused by SQL; it reflects the **one-to-many relationship**.

This is an important thing to understand when debugging query results.

13. GROUP BY

`GROUP BY` combines rows into groups so aggregate functions can be applied.

Suppose:

payment_id	customer_id	amount	status

P1	C101	1000	SUCCESS
P2	C101	2000	FAILED
P3	C102	5000	SUCCESS

To calculate total successful payment amount per customer:

```
SELECT
    customer_id,
    SUM(amount) AS total_amount
FROM payment
WHERE status = 'SUCCESS'
GROUP BY customer_id;
```

Result:

customer_id	total_amount
C101	1000
C102	5000

14. Aggregate Functions

Know these:

```
COUNT()
SUM()
AVG()
MIN()
MAX()
```

Examples:

```
SELECT COUNT(*)
FROM payment;
```

```
SELECT SUM(amount)
FROM payment
WHERE status = 'SUCCESS';
```

```
SELECT MAX(amount)
FROM payment;
```

15. GROUP BY Multiple Columns

You can group by more than one column:

```
SELECT
    customer_id,
    status,
```

```
COUNT(*) AS payment_count
FROM payment
GROUP BY customer_id, status;
```

Conceptually:

```
C101 + SUCCESS
C101 + FAILED
C102 + SUCCESS
```

Each combination becomes a group.

16. HAVING

WHERE filters **rows before grouping**.

HAVING filters **groups after grouping**.

Example:

Find customers whose successful payment total exceeds 10,000.

```
SELECT
    customer_id,
    SUM(amount) AS total_amount
FROM payment
WHERE status = 'SUCCESS'
GROUP BY customer_id
HAVING SUM(amount) > 10000;
```

Remember

```
WHERE
→ filter rows

GROUP BY
→ create groups

HAVING
→ filter groups
```

17. Very Common Interview Question

WHERE vs **HAVING**

Wrong:

```
WHERE SUM(amount) > 10000
```

Aggregate filtering belongs in **HAVING** after grouping.

Correct:

```
HAVING SUM(amount) > 10000
```

18. SQL Logical Execution Order

This helps explain why `WHERE` and `HAVING` behave differently.

Conceptually:

```
FROM
↓
JOIN
↓
WHERE
↓
GROUP BY
↓
HAVING
↓
SELECT
↓
ORDER BY
↓
LIMIT
```

This is a **logical processing order**, not necessarily the physical execution order chosen by the database optimizer.

For interviews, knowing this distinction is useful.

19. Subqueries

A **subquery** is a query inside another query.

Example:

Find payments greater than the average payment.

```
SELECT payment_id, amount
FROM payment
WHERE amount > (
    SELECT AVG(amount)
    FROM payment
);
```

Inner query:

```
SELECT AVG(amount)
FROM payment;
```

returns one value.

Outer query uses it.

20. Scalar Subquery

A subquery that returns a single value.

```
SELECT *
FROM payment
WHERE amount > (
    SELECT AVG(amount)
    FROM payment
);
```

The inner query returns one value:

```
2500
```

Then outer query becomes conceptually:

```
WHERE amount > 2500
```

21. Subquery with IN

Suppose you want customers who have successful payments.

```
SELECT *
FROM customer
WHERE customer_id IN (
    SELECT customer_id
    FROM payment
    WHERE status = 'SUCCESS'
);
```

Inner query produces:

```
C101
C102
```

Outer query returns those customers.

22. EXISTS

Another very important form.

Find customers who have at least one successful payment.

```
SELECT
    c.customer_id,
    c.name
FROM customer c
WHERE EXISTS (
    SELECT 1
```

```
FROM payment p
WHERE p.customer_id = c.customer_id
AND p.status = 'SUCCESS'
);
```

EXISTS checks whether **at least one matching row exists**.

You don't actually care about the selected value, which is why:

```
SELECT 1
```

is commonly written.

23. EXISTS vs IN

They can produce similar results, but they have different semantics and can behave differently depending on data, nulls, and the optimizer.

Think:

```
IN
→ Is this value present in this result set?

EXISTS
→ Does at least one matching row exist?
```

For correlated existence checks, **EXISTS** can express the intent very clearly.

Don't claim one is universally faster; modern database optimizers can transform queries in sophisticated ways.

24. Correlated Subquery

A correlated subquery refers to a column from the outer query.

Example:

```
SELECT
  c.customer_id,
  c.name
FROM customer c
WHERE EXISTS (
  SELECT 1
  FROM payment p
  WHERE p.customer_id = c.customer_id
);
```

Here:

```
p.customer_id = c.customer_id
```

connects the inner query to the current outer-row customer.

That's a **correlated subquery**.

25. Subquery in FROM

A subquery can also behave like a derived table.

Example:

```
SELECT
  x.customer_id,
  x.total_amount
FROM (
  SELECT
    customer_id,
    SUM(amount) AS total_amount
  FROM payment
  GROUP BY customer_id
) x
WHERE x.total_amount > 10000;
```

Inner query:

```
customer → total amount
```

Outer query:

```
filter totals > 10000
```

26. JOIN vs Subquery

This is a common design question.

Suppose:

Get customers who have successful payments.

Using JOIN:

```
SELECT DISTINCT c.customer_id, c.name
FROM customer c
JOIN payment p
  ON p.customer_id = c.customer_id
WHERE p.status = 'SUCCESS';
```

Using EXISTS :

```
SELECT c.customer_id, c.name
FROM customer c
WHERE EXISTS (
  SELECT 1
  FROM payment p
  WHERE p.customer_id = c.customer_id
  AND p.status = 'SUCCESS'
);
```

Both can express the requirement.

Which is better?

There is no universal answer.

Consider:

- readability
- result shape
- whether you need columns from both tables
- cardinality
- indexes
- execution plan
- database optimizer

27. Important JOIN Mistake

Suppose you want **all customers**, including those without payments:

```
SELECT c.name, p.amount
FROM customer c
LEFT JOIN payment p
  ON c.customer_id = p.customer_id
WHERE p.status = 'SUCCESS';
```

This can effectively eliminate customers where `p` is `NULL`, making the result behave much more like an inner join.

If the requirement is:

all customers, but only match successful payments

you might instead put the condition in the `ON` clause:

```
SELECT c.name, p.amount
FROM customer c
LEFT JOIN payment p
  ON c.customer_id = p.customer_id
  AND p.status = 'SUCCESS';
```

This distinction is **very important** in SQL interviews.

28. JOIN with GROUP BY

A realistic query:

Find each customer's successful payment count and total amount.

```
SELECT
```

```

        c.customer_id,
        c.name,
        COUNT(p.payment_id) AS payment_count,
        COALESCE(SUM(p.amount), 0) AS total_amount
FROM customer c
LEFT JOIN payment p
    ON c.customer_id = p.customer_id
   AND p.status = 'SUCCESS'
GROUP BY
    c.customer_id,
    c.name;

```

This gives all customers, including customers with zero successful payments.

`COALESCE()` converts `NULL` total to `0`.

29. A Very Common Interview Query

Find duplicate transaction IDs

Suppose:

```

payment
transaction_id

```

should be unique.

```

SELECT
    transaction_id,
    COUNT(*) AS count
FROM payment
GROUP BY transaction_id
HAVING COUNT(*) > 1;

```

This combines:

```

GROUP BY
+
COUNT
+
HAVING

```

You should know this.

30. Another Common Query

Find the highest payment

```

SELECT MAX(amount)
FROM payment;

```

Second-highest distinct amount

One approach:

```
SELECT MAX(amount)
FROM payment
WHERE amount < (
    SELECT MAX(amount)
    FROM payment
);
```

This is a classic subquery problem.

31. Another Important Query

Customers with no payments

```
SELECT c.customer_id, c.name
FROM customer c
LEFT JOIN payment p
    ON c.customer_id = p.customer_id
WHERE p.payment_id IS NULL;
```

Know this pattern very well.

32. SQL NULL

Interviewers frequently test this.

You cannot correctly write:

```
WHERE payment_id = NULL
```

Use:

```
WHERE payment_id IS NULL
```

or:

```
WHERE payment_id IS NOT NULL
```

Because `NULL` means **unknown/missing**, not a normal value.

33. Query Your Database Like a Backend Developer

At 3 years, don't just write SQL—you should understand how the query maps to Java code.

For example:

```
List<Payment> payments =
    paymentRepository.findByCustomerId(customerId);
```

Under the hood, the database executes SQL.

You should be able to recognize:

```
Java/JPA
↓
Generated SQL / native query
↓
Database
↓
Result set
↓
Entity mapping
```

This becomes important later when learning **JPA/Hibernate performance**.

34. SQL Questions You Should Be Ready For

SELECT

1. What does `SELECT` do?
2. Why avoid `SELECT *` ?
3. `WHERE` vs `HAVING` ?
4. `ORDER BY` ?
5. `DISTINCT` ?

JOIN

6. INNER JOIN?
7. LEFT JOIN?
8. RIGHT JOIN?
9. INNER vs LEFT JOIN?
10. Why would a LEFT JOIN unexpectedly behave like an INNER JOIN?
11. What happens when a one-to-many table is joined?
12. How do you find rows with no matching record?

GROUP BY

13. What is GROUP BY?
14. What are aggregate functions?
15. Why do we need HAVING?
16. WHERE vs HAVING?
17. GROUP BY multiple columns?

Subqueries

18. What is a subquery?
19. Scalar subquery?
20. `IN` vs `EXISTS` ?

- 21. What is a correlated subquery?
- 22. Can a subquery appear in FROM?
- 23. JOIN vs subquery?

35. 3-Year-Level Questions

Be ready for these:

Q1. Find customers whose total successful payments exceed ₹10,000.

```
SELECT
    customer_id,
    SUM(amount) AS total_amount
FROM payment
WHERE status = 'SUCCESS'
GROUP BY customer_id
HAVING SUM(amount) > 10000;
```

Q2. Find customers who have never made a payment.

```
SELECT c.customer_id, c.name
FROM customer c
LEFT JOIN payment p
    ON c.customer_id = p.customer_id
WHERE p.payment_id IS NULL;
```

Q3. Find duplicate transactions.

```
SELECT transaction_id, COUNT(*)
FROM payment
GROUP BY transaction_id
HAVING COUNT(*) > 1;
```

Q4. Find payments above the average payment amount.

```
SELECT *
FROM payment
WHERE amount > (
    SELECT AVG(amount)
    FROM payment
);
```

Q5. Find customers with at least one successful payment.

```
SELECT c.customer_id, c.name
FROM customer c
```

```
WHERE EXISTS (  
  SELECT 1  
  FROM payment p  
  WHERE p.customer_id = c.customer_id  
  AND p.status = 'SUCCESS'  
);
```

36. How the Interviewer May Go Deeper

They might ask:

“This query works. How would you make it faster?”

Don't immediately rewrite the SQL.

Think:

1. How much data?
2. What filters?
3. What joins?
4. Which columns are indexed?
5. What is the execution plan?
6. How many rows are being scanned?
7. Are we selecting unnecessary columns?
8. Is there a better query shape?
9. Is pagination needed?

That leads naturally into your next SQL topics:

indexes → query execution plans → transactions → isolation → optimization.

37. The SQL Mental Model

```
SELECT  
↓  
FROM  
↓  
JOIN  
↓  
WHERE  
↓  
GROUP BY  
↓  
HAVING  
↓  
ORDER BY  
↓  
LIMIT
```

For relationships:

```
INNER JOIN  
→ matching records  
  
LEFT JOIN  
→ all left + matching right
```

RIGHT JOIN
→ all right + matching left

For aggregation:

GROUP BY
→ create groups

HAVING
→ filter groups

For nested logic:

Subquery
→ query inside another query

38. What You Should Know at 3 Years

Must know

- SELECT
- WHERE
- ORDER BY
- DISTINCT
- INNER JOIN
- LEFT JOIN
- RIGHT JOIN
- GROUP BY
- COUNT , SUM , AVG , MIN , MAX
- HAVING
- subqueries
- IN
- EXISTS
- correlated subquery
- NULL
- duplicate detection
- no-match queries

Next level

- indexes
- composite indexes
- execution plans
- CTE
- window functions
- pagination
- query optimization
- transactions

- isolation levels
- deadlocks

For your **3-year backend preparation**, the next SQL topic after this should be **Indexes + query execution plans**, because interviewers often follow a SQL query question with: **“This query is slow in production. How would you investigate and optimize it?”**